Revision Exercise
Review Questions

Total points: 100

The **Cosmetic Sales Management Application** helps a cosmetic store track its stock. Users can view all cosmetics, add a new cosmetic, export (sell) cosmetics, delete a cosmetic, view low-stock warnings, and view an inventory summary. All data is stored in a SQLite database. The application must **calculate** the stock status and the total inventory value automatically.

Data Structure

Each cosmetic is stored in a SQLite table named `cosmetics` with the following fields:

- `id INTEGER PRIMARY KEY AUTOINCREMENT`: unique identifier of the cosmetic.
- `name TEXT NOT NULL`: name of the cosmetic (e.g., "Son Mac Ruby Woo").
- `category TEXT`: category (e.g., `Lipstick`, `Foundation`, `Skincare`).
- `price REAL NOT NULL`: selling price per unit in VND (e.g., 650000).
- `stock INTEGER NOT NULL`: quantity currently in stock, in units (e.g., 50).

Business Note: the **Stock Status** is derived automatically from `stock` and **cannot** be edited manually: `Out of stock` if stock $= 0$; `Low stock` if $1 \leq$ stock ≤ 9; `In stock` if stock ≥ 10. A cosmetic is *low stock* when stock ≤ 9.

General Requirements

- **Project folder**: must be named `fa26_prm393_StudentCode` (e.g., `fa26_prm393_ce123456`).
- **Development tool**: use Android Studio as your development IDE.
- **Technology**: use Flutter for the mobile app and SQLite for database management.
- **Data**: data must be stored and managed using SQLite.
- **State management**: the global cosmetic-list state must be managed using **Provider**.

Tasks

1. **Cosmetic List Screen** Display all cosmetics using a `ListView`, including Name, Category, Price, Stock, an automatically computed **Stock Status**, and an **inventory summary**. [35 points]
   - Fetch the list from the SQLite database. If the list is empty, show a simple empty-state message. Provide a "+" `FloatingActionButton` that navigates to the Add cosmetic screen. (5)
   - Display each cosmetic in a `ListTile` with `Icons.brush` as the leading icon, the name as the title, and "Category – Price – Stock" as the subtitle. Provide `Icons.edit` and `Icons.delete` as trailing actions (Edit opens the Export screen in Task 3; Delete follows Task 4). (10)
   - Compute and display the **Stock Status** per cosmetic with a distinct color/indicator. (10)
   - Display an **inventory summary**: **Total inventory value** $= \sum(\text{price} \times \text{stock})$ over all cosmetics (thousand separators), plus the number of out-of-stock and low-stock cosmetics. Recalculate whenever the data changes. (10)
2. **Add New Cosmetic Screen** Create the Add Screen with the following fields: Name, Category, Price, Stock. [15 points]

- Validate that all inputs are required. Price must be a positive number and Stock must be a non-negative integer. (5)
- The price and stock FormFields use the number keyboard type. (5)
- After the user taps "Save", the new cosmetic is saved to the SQLite database, the Stock Status is derived automatically, and the app routes back to the List Screen and refreshes the data and the inventory summary immediately. (5)

3. **Export (Sell) cosmetics** When the user taps the `Icons.edit` icon of a cosmetic, open an **Export** form to sell a quantity of that cosmetic. [20 points]
   - The form shows the cosmetic name and current stock, and lets the user enter a quantity to export. (5)
   - Validate that the quantity is a positive integer and does not exceed the current stock ($0 <$ qty $\leq$ stock). (5)
   - After the user confirms, deduct the quantity from `stock` in the SQLite database, recalculate the Stock Status and the inventory summary, and refresh the List Screen. If `stock` becomes ≤ 9, the item must appear in the Low Stock Warning Screen. (10)
4. **Delete a cosmetic** Create the Delete confirmation dialogue. [10 points]
   - After the user taps the "Delete" icon, open a dialogue to confirm the action, displaying the chosen cosmetic's name. (5)
   - After the user taps "Delete", the cosmetic is removed from the SQLite database and the List Screen (and summary) is refreshed. (5)
5. **Low Stock Warning Screen** Add an AppBar action (`Icons.warning_amber`) that opens a dedicated screen listing cosmetics that are about to run out. [10 points]
   - List all cosmetics with `stock` ≤ 9 (both `Low stock` and `Out of stock`) using a `ListView`, showing Name, Stock, and Status. (5)
   - Display the total number of low-stock cosmetics. The list must reflect the latest stock after any Add / Export / Delete action. (5)
6. **Correct practices with Dart/Flutter development and UI respect** [10 points]
   - Respect UI suggestions: AppBar titles, color is rose, components displayed without overflow. (5)
   - Dispose the owned controllers after use, use correct OOP, and apply consistent theming. (5)

Sample Data (Manual Testing)

Enter each record through the Add cosmetic screen. ID is auto-generated; Stock Status is derived automatically.

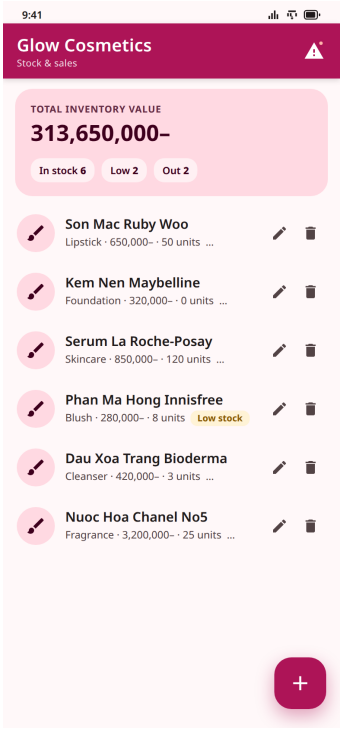
Id	Name	Category	Price	Stock	Status
1	Son Mac Ruby Woo	Lipstick	650,000	50	In stock
2	Kem Nen Maybelline	Foundation	320,000	0	Out of stock
3	Serum La Roche-Posay	Skincare	850,000	120	In stock
4	Phan Ma Hong Innisfree	Blush	280,000	8	Low stock
5	Dau Xoa Trang Bioderma	Cleanser	420,000	3	Low stock
6	Nuoc Hoa Chanel No5	Fragrance	3,200,000	25	In stock
7	Chi Ke Mat 3CE	Eyeliner	250,000	0	Out of stock
8	Kem Chong Nang Anessa	Sunscreen	520,000	40	In stock
9	Son Duong Laneige	Lip Balm	390,000	15	In stock
10	Mat Na Sulwhasoo	Mask	1,150,000	60	In stock

Expected results after entering all 10 records:

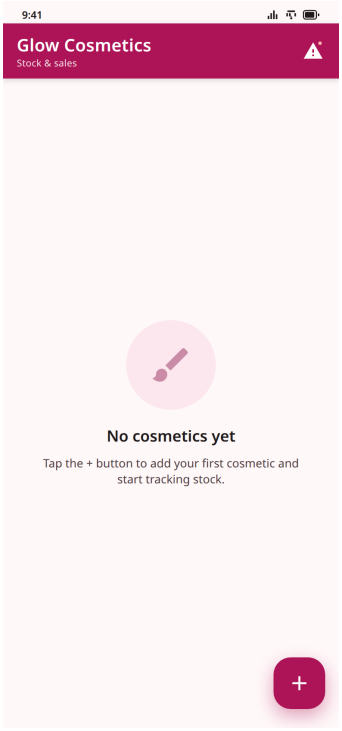
- Total cosmetics: 10 | Out of stock: 2 | Low stock: 2 | In stock: 6
- Low Stock Warning Screen lists **4** cosmetics (ids 2, 4, 5, 7).
- Total inventory value: **313,650,000 VND**

Illustrations

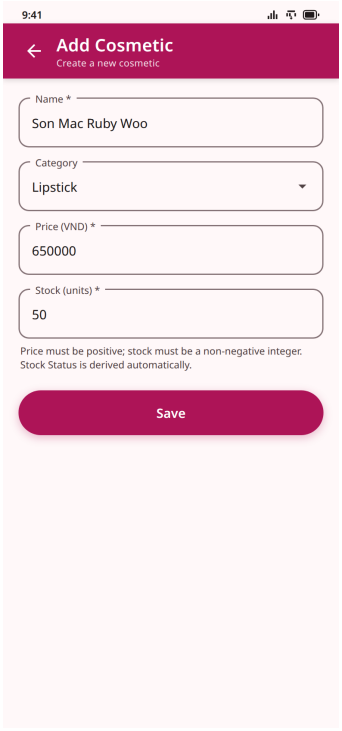
The following screens illustrate the expected Material 3 user interface for each task. Colors are rose based on the cosmetic theme.



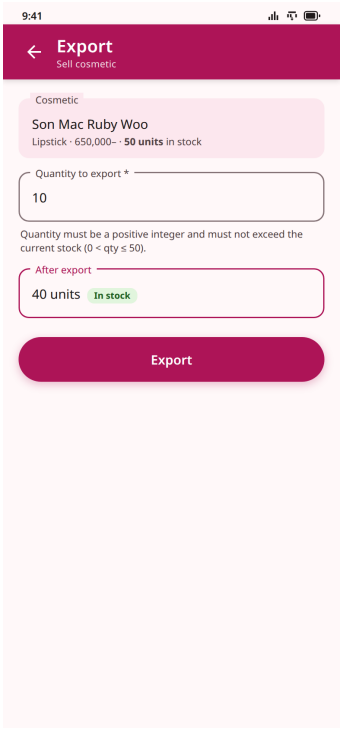
(a) Cosmetic List



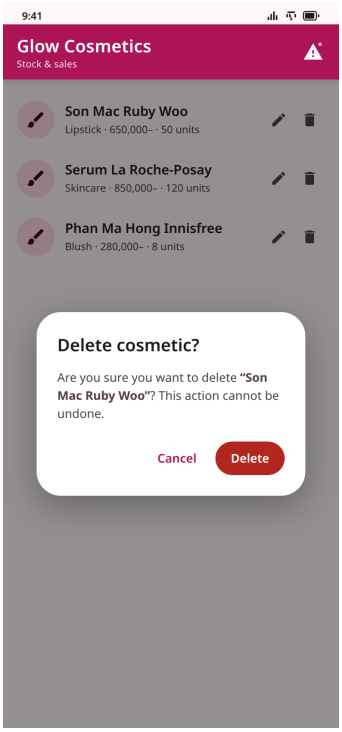
(b) Empty state



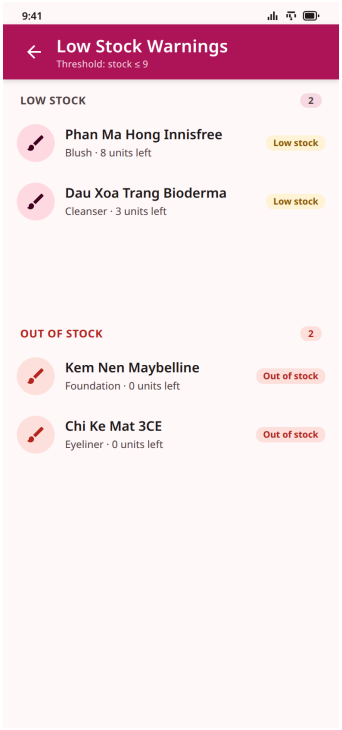
(c) Add Cosmetic



(d) Export (Sell)



(e) Delete confirmation



(f) Low Stock Warnings

- Students may use compatible Flutter/Dart versions and offline resources available on their computers.
- Run `flutter clean` before submitting the project folder. No compression is required.
- Students are **NOT ALLOWED** to use any Internet resources such as chat programs, email, remote desktop, and especially AI tools. These actions are considered cheating and will receive zero points.
- **Rules**: the project will receive zero points if it fails to run, or if it contains data structures, variables, images, screens, or UI elements unrelated to the examination topic.

— THE END —